

ClaimSetu — Technical Architecture & System Design Report

Prepared For: Senior Review

Project: ClaimSetu — Har Claim Ka Saathi

Prepared By: Development Team

1. Project Overview

ClaimSetu ek Indian legal-tech + fintech automation platform hai jo death claim paperwork ko simplify karta hai.

Core Problem

India mein kisi person ki death hone ke baad:

- Bank accounts freeze ho jaate hain
- Har bank ka alag claim process hota hai
- Families ko multiple forms bharne padte hain
- Lawyers/agents ₹5,000-₹15,000 charge karte hain
- Process bahut slow aur stressful hota hai

Solution

ClaimSetu user se ek baar data collect karega aur:

- Different bank claim forms auto-generate karega
 - PDF documents create karega
 - Checklist aur next steps dega
 - WhatsApp sharing support karega
-

2. Proposed Technology Stack

Layer	Technology
Mobile App	React Native CLI
Admin Panel	Next.js
Backend API	Express.js

Layer	Technology
Database	PostgreSQL / Supabase
Storage	Supabase Storage
PDF Engine	pdf-lib
Monorepo	Turborepo + pnpm
Validation	Zod
Shared Types	TypeScript

3. Why Full JavaScript Stack?

Advantages

1. Shared Ecosystem

- Same language across frontend/backend
- Faster development
- Shared utilities
- Shared validation logic

2. Monorepo Friendly

- Shared types
- Shared business logic
- Shared validation schemas
- Better maintainability

3. Faster MVP Development

- Single developer productivity high
- Easier onboarding
- Faster iteration cycle

4. Strong PDF Ecosystem

Node.js ecosystem has excellent PDF support:

- pdf-lib
 - pdfkit
 - puppeteer
 - pdfme
-

4. Recommended Monorepo Architecture

```
claimsetu/
├── apps/
│   ├── mobile/      # React Native CLI
│   ├── admin/       # Next.js
│   └── backend/     # Express.js
├── packages/
│   ├── types/
│   ├── validations/
│   ├── utils/
│   ├── config/
│   └── pdf-engine/
├── turbo.json
├── pnpm-workspace.yaml
└── package.json
```

5. Why Monorepo?

Benefits

Shared Types

Example:

```
export type Claim = {
  deceasedName: string;
  aadhaar: string;
}
```

Same type:

- mobile app
- admin panel
- backend API

sab use kar sakte hain.

Shared Validation

Using Zod:

```
export const claimSchema = z.object({
  deceasedName: z.string(),
  aadhaar: z.string(),
});
```

Validation consistency maintain hoti hai.

Shared PDF Logic

PDF engine reusable hoga.

Example:

```
packages/pdf-engine/
```

Contains:

- coordinate utilities
 - PDF loader
 - field renderer
 - export functions
-

6. System Architecture

High-Level Flow

```
Mobile App
↓
Backend API
↓
Supabase/PostgreSQL
↓
PDF Engine
↓
Supabase Storage
↓
Generated PDFs
↓
User Download/Share
```

7. User App Flow

Main Workflow

```
Splash Screen
↓
Authentication
↓
Home Dashboard
↓
Select Bank
↓
Select Claim Type
↓
Form Wizard
↓
Payment
↓
PDF Generation
↓
Claim Ready Screen
↓
Track Claim
```

8. Form Wizard Structure

Step 1 — Deceased Details

Fields:

- Full Name
 - Date of Birth
 - Date of Death
 - Aadhaar
 - PAN
 - Address
 - Bank Account Details
-

Step 2 — Legal Heirs

Fields:

- Claimant Name
- Relationship

- Aadhaar
 - PAN
 - Bank Account
 - Share Percentage
-

Step 3 — Witness Details

Fields:

- Witness Name
 - Aadhaar
 - Mobile
 - Address
-

Step 4 — Document Upload

Documents:

- Death Certificate
 - Aadhaar
 - PAN
 - Passbook
 - Cancelled Cheque
-

Step 5 — Review

- All details verification
 - Edit option
 - Confirm and proceed
-

9. PDF Generation Architecture

Core Idea

System:

- original bank PDF template use karega
 - user data overlay karega
 - final generated PDF export karega
-

10. PDF Generation Workflow

```
User Data
↓
Fetch PDF Template
↓
Fetch Field Mappings
↓
Apply Coordinates
↓
Draw Text
↓
Generate Final PDF
↓
Store in Cloud
```

11. Why Template-Based PDF Generation?

Advantages

1. Exact Bank Format

Original bank PDF maintain hota hai.

2. Faster Development

Custom PDF designing avoid hoti hai.

3. Legal Safety

Bank-approved format same rehta hai.

4. Easy Scaling

Sirf:

- new PDF upload
- mappings create

karna hota hai.

12. PDF Field Mapping System

Core Concept

Every PDF form ka:

- alag template
- alag field coordinates

hoga.

13. Mapping Table Design

```
CREATE TABLE pdf_field_mappings (  
  id UUID PRIMARY KEY,  
  bank_form_id UUID,  
  field_key TEXT,  
  x_position INT,  
  y_position INT,  
  page_number INT,  
  font_size INT DEFAULT 12  
);
```

14. Mapping Example

SBI Main Form

field_key	x	y
deceased_name	120	340
aadhaar	120	410

HDFC Form

field_key	x	y
deceased_name	180	420

Same data field different forms mein different coordinates pe render hoga.

15. PDF Generation Logic

Backend workflow:

```
for (const field of mappings) {  
  
  const value = claimData[field.field_key];  
  
  page.drawText(value, {  
    x: field.x_position,  
    y: field.y_position,  
    size: field.font_size  
  });  
  
}
```

16. Why Admin Panel is Critical?

Admin panel is:

Document Automation Control Center

Without admin panel:

- every new bank requires coding
- every form update manual hoga
- scalability difficult ho jayegi

17. Admin Panel Responsibilities

1. Bank Management

- Add/Edit Banks
- Upload logos
- Configure claim types

2. PDF Template Upload

Admin actual bank forms upload karega.

Example:

- SBI Main Form
- SBI Affidavit
- HDFC Claim Form

3. Visual Field Mapping

Admin:

- PDF open karega
- blank field pe click karega
- database field select karega
- mapping save karega

No coding required.

4. Rule Engine Management

Example:

```
IF amount > 5 lakh  
THEN add surety form
```

OR

```
IF locker_claim = true  
THEN add locker inventory
```

18. Visual Mapping Tool Architecture

Recommended Stack

Purpose	Tech
PDF Viewer	react-pdf
Zoom/Pan	react-zoom-pan-pinch
UI	Next.js + Tailwind
Coordinate Capture	Mouse Events

19. Mapping Tool Flow

```
Upload PDF
↓
Open PDF Preview
↓
Click Blank Area
↓
Select Field Name
↓
Save Coordinates
```

20. Backend Architecture

Recommended Structure

```
src/

modules/
  claims/
  pdf/
  auth/
  banks/
  payments/
  notifications/
```

21. Why Express.js?

Advantages

- Fast MVP development
 - Huge ecosystem
 - Flexible architecture
 - Excellent PDF libraries
 - Strong community support
-

Limitations

- No enforced architecture
- Scaling can become messy
- Dependency Injection missing

- Large projects require discipline
-

22. Recommendation Regarding Express.js

Short-Term (MVP)

Express.js is acceptable and fast.

Long-Term

Migration path to NestJS should remain open.

Suggested approach:

- use Express.js initially
 - maintain modular architecture
 - avoid monolithic files
-

23. Database Recommendation

Supabase PostgreSQL

Reasons:

- PostgreSQL reliability
 - built-in auth
 - storage support
 - realtime features
 - scalable
 - easy integration
-

24. Storage Architecture

Templates

```
templates/  
  SBI/  
    main_form.pdf  
    affidavit.pdf
```

Generated PDFs

```
generated/  
claim_123/
```

25. Future Scalability

Same architecture later support karega:

- Insurance Claims
 - PF Claims
 - Mutual Funds
 - Demat Transfer
 - Property Transfer
 - Estate Locker
-

26. Security Considerations

Required

- Row Level Security
 - Secure Storage
 - Signed URLs
 - Role-based admin access
 - Audit logs
 - Encrypted sensitive data
-

27. Recommended Development Phases

Phase 1 — MVP

- Authentication
 - Form Wizard
 - Basic PDF generation
 - SBI support
 - Storage integration
-

Phase 2

- Admin mapping tool
- Multiple banks

- Payment integration
 - Claim tracking
-

Phase 3

- AI/OCR
 - Smart rules engine
 - Partner dashboards
 - Automation workflows
-

28. Final Recommendation

Recommended Architecture

```
Monorepo → Turborepo  
  
Mobile → React Native CLI  
Admin → Next.js  
Backend → Express.js  
DB → Supabase PostgreSQL  
Storage → Supabase Storage  
PDF → pdf-lib
```

29. Final Conclusion

ClaimSetu is not just a form-filling app.

It is:

- document automation platform
- workflow engine
- legal-tech SaaS
- fintech process automation system

The proposed architecture:

- scalable
- maintainable
- production-ready
- suitable for rapid MVP development
- future extensible

Main scalability strength:

Template + Mapping + Rule Engine architecture

This allows:

- adding new banks without coding
- dynamic PDF generation
- centralized control via admin panel
- long-term SaaS scalability

30. Detailed Database Review & Recommendations

Existing Database Evaluation

Current schema architecture is good for:

- MVP launch
- multi-bank support
- claim tracking
- PDF generation foundation
- scalable SaaS structure

However, production-grade legal-tech automation ke liye kuch critical tables aur improvements required hain.

31. Current Tables Review

Table	Status	Notes
users	Good	User profile structure correct
banks	Good	Proper master table
bank_forms	Needs improvement	Template/version separation needed
claims	Good	Core workflow table
deceased_details	Good	Proper claim linkage
legal_heirs	Good	Multi-heir support excellent
witnesses	Good	Suitable for banking workflow
claim_documents	Good	Needs document status improvements
generated_pdfs	Good	Strong audit capability
payments	Good	Needs gateway response storage

Table	Status	Notes
claim_timeline	Excellent	Good workflow tracking
notifications	Good	Notification support ready

32. Most Important Missing Table — pdf_field_mappings

Why This Table Is Critical

This is the CORE of ClaimSetu automation engine.

Without this table:

- dynamic PDF generation difficult ho jayega
- every bank form hardcoded ho jayega
- scalability break ho jayegi

Recommended Table

```
CREATE TABLE pdf_field_mappings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  bank_form_id UUID REFERENCES bank_forms(id) ON DELETE CASCADE,

  field_key TEXT NOT NULL,

  x_position FLOAT NOT NULL,
  y_position FLOAT NOT NULL,

  page_number INT DEFAULT 1,

  font_size INT DEFAULT 12,

  font_family TEXT DEFAULT 'Helvetica',

  text_color TEXT DEFAULT '#000000',

  max_width FLOAT,

  is_multiline BOOLEAN DEFAULT false,

  alignment TEXT DEFAULT 'left',
```



```
created_at TIMESTAMPTZ DEFAULT now()
);
```

33. Purpose of pdf_field_mappings

This table stores:

- which field to render
- where to render
- on which page
- in which style

Example:

field_key	x	y
deceased_name	120	340
aadhaar_number	120	410

Different forms:

- different coordinates
- same backend engine

34. Recommended form_rules Table

Why Needed?

Different banks different conditions use karte hain.

Example:

- amount > 5 lakh
- locker claim
- nominee exists

Forms dynamically include/exclude honge.

Recommended Table

```
CREATE TABLE form_rules (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

bank_form_id UUID REFERENCES bank_forms(id),

condition_key TEXT,
operator TEXT,
condition_value TEXT,

action_type TEXT,
action_value TEXT,

created_at TIMESTAMPTZ DEFAULT now()
);

```

Example Rules

condition	action
amount > 5 lakh	add surety form
locker=true	add locker inventory
nominee_exists=true	skip affidavit

35. Recommended form_templates Table

Why Needed?

Banks future mein forms update karenge. Versioning required hoga.

Recommended Table

```

CREATE TABLE form_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  bank_form_id UUID REFERENCES bank_forms(id),

  template_name TEXT,

  file_url TEXT,

  pages INT,

  version TEXT,

```

```
is_active BOOLEAN DEFAULT true,  
  
created_at TIMESTAMPTZ DEFAULT now()  
);
```

36. Recommended audit_logs Table

Why Important?

Legal-tech systems mein audit tracking mandatory hota hai.

Track:

- PDF generation
- claim edits
- document deletion
- payment updates

Recommended Table

```
CREATE TABLE audit_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  user_id UUID,  
  
  action TEXT,  
  
  entity_type TEXT,  
  
  entity_id UUID,  
  
  metadata JSONB,  
  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

37. Recommended admin_users Table

Why Needed?

Admin panel access separate hona chahiye.

Recommended Table

```
CREATE TABLE admin_users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  email TEXT UNIQUE,  
  
  full_name TEXT,  
  
  role TEXT,  
  
  is_active BOOLEAN DEFAULT true,  
  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

38. Recommended Improvements to Existing Tables

claims Table

Add:

```
claim_amount NUMERIC,  
metadata JSONB
```

Reason:

- rule engine
- future extensibility

legal_heirs Table

Add:

```
is_nominee BOOLEAN DEFAULT false
```

Reason:

- nominee-based workflows

claim_documents Table

Add:

```
document_status TEXT DEFAULT 'uploaded'
```

Values:

- uploaded
- verified
- rejected

payments Table

Add:

```
payment_gateway_response JSONB
```

Reason:

- debugging
- payment audits

generated_pdfs Table

Add:

```
generation_status TEXT DEFAULT 'pending'
```

39. Index Recommendations

Why Important?

Without indexes: large-scale queries slow ho jayengi.

Recommended Indexes

```
CREATE INDEX idx_claims_user_id  
ON claims(user_id);
```

```
CREATE INDEX idx_claim_documents_claim_id  
ON claim_documents(claim_id);
```

```
CREATE INDEX idx_legal_heirs_claim_id  
ON legal_heirs(claim_id);
```

```
CREATE INDEX idx_generated_pdfs_claim_id  
ON generated_pdfs(claim_id);
```

40. Storage Architecture Recommendation

Templates

```
templates/  
  SBI/  
  HDFC/  
  ICICI/
```

Generated PDFs

```
generated/  
  claim_123/
```

Uploaded Documents

```
documents/  
  claim_123/
```

41. Recommended Final Database Architecture

Core Tables

User Layer

- users
- admin_users

Claim Layer

- claims
- deceased_details
- legal_heirs
- witnesses

Bank Layer

- banks
- bank_forms
- form_templates
- form_rules

PDF Engine Layer

- pdf_field_mappings
- generated_pdfs

Upload Layer

- claim_documents

Financial Layer

- payments

Tracking Layer

- claim_timeline
- notifications
- audit_logs

42. Final Recommendation

Current database structure is a strong foundation.

However, for real production-ready document automation:

MUST ADD:

- pdf_field_mappings
- form_rules
- form_templates
- audit_logs
- indexes

These tables are critical for:

- scalable PDF automation
- dynamic bank support
- admin-controlled mappings
- future SaaS scaling
- maintainability

43. Final Verdict

Current Schema Status

Category	Rating
MVP Readiness	9/10
Scalability	8.5/10
Legal-Tech Readiness	8/10
PDF Automation Readiness	6.5/10
Future Extensibility	8.5/10

After Recommended Improvements

Category	Rating
MVP Readiness	9.5/10
Scalability	9.5/10
Legal-Tech Readiness	9.5/10
PDF Automation Readiness	10/10
Future Extensibility	9.5/10

End of Report